Syamasudha Veeragandham

23BCE0722_23BCE2236_23BCE2206_Advanced_Java_Progra...

Student Project

Document Details

Submission ID
trn:oid:::3618:155965604

Submission Date
Oct 26, 2026, 7:38 PM GMT+5:30

Download Date
Oct 26, 2026, 7:42 PM GMT+5:30

File Name
23BCE0722_23BCE2236_23BCE2206_Advanced_Java_Programming_Project_to_rewrite_REVISED1.docx

File Size
3.1 MB

23 Pages

7,722 Words

41,395 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Database Query Visual Simulator: An Interactive What-If Sandbox for SQL Query Cost Explanation

Divyaraj[1], Joel Devasia[2], Alen Varghese Soji[3] and Syamasudha Veeragandham[4]

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014

Email: divyaraj.2023@vitstudent.ac.in, joel.devasia2023@vitstudent.ac.in, alenvarghese.soji2023@vitstudent.ac.in, syamasudha.v@vit.ac.in

[4] Corresponding author

Abstract

Writing SQL is quite easy but to figure out why it's slow is where most people can get confused on. Even students, junior developers and DBAs who've been doing this for years can struggle to find performance in their queries because the standard tool for the job that is EXPLAIN just throws operator names and cost numbers with no context and no visual explanation attached to any of it. Testing a fix is a problem on it's own too because nobody wants to insert a new index onto a live production table just to see what happens, so a lot of prior work ends up sidestepping the problem rather than solving it. Indexing, partitioning, and query rewriting all do work traditionally speaking but someone still has to apply them manually and the newer ML and reinforcement learning optimizers trade that manual effort for an all new kind cost altogether - they need training data and live statistics which most projects don't have yet. This paper presents the Database Query Visual Simulator, a web tool that takes a raw SQL query and a hypothetical schema, parses them with ANTLR4, estimates cost using a PostgreSQL-style heuristic model in a Java backend, and draws the resulting execution plan as an interactive graph. Flip a hypothetical index on for any column and the cost updates right away all with no live database and no real data that can get affected.

Keywords

SQL Query Optimization, Query Execution Plan Visualization, Cost-Based Query Optimization, What-If Index Analysis, ANTLR-Based SQL Parsing, Interactive Node-Graph Visualization, Database Explainability

1. Introduction

Most data-heavy applications still run on relational databases, and how fast they feel usually comes down to how well their SQL performs, and that's been true for decades; it just matters more now that "a lot of data" means millions or billions of rows instead of thousands. A query doesn't run exactly as written. It first passes through a query optimizer, which has to decide, quickly, which of many possible ways of fetching the data will be cheapest: sequential scan or index scan, nested-

loop join or hash join or merge join, and once you're past two tables, which order to join them in. The optimizer scores each option with a cost model built from estimated row counts, selectivity, and I/O and CPU overhead, and picks whatever comes out cheapest. None of this is a minor implementation detail either, since a good plan can run an order of magnitude faster than a bad one, and at scale that's the difference between a query that returns instantly and one that stalls the whole application. That's also why query optimization has stayed such an active research area, pulling in new heuristic, cost-based, and now machine-learning approaches over the years [1]–[10].

When a programmer or DBA is asked why a query runs slowly, the first thing to do is to perform the EXPLAIN and look at the results, the name of the operators, many cost and estimated numbers of rows, and converted to follow the plan's branches in plain text form. A simple query containing two or more joins and a filter will produce many lines. Reading them correctly requires knowledge in advance about which of the operations are expensive, why the planner preferred this or that order of joins, and how the costs are related. It is quite well known that a person who does not know what to do with the output normally so they then turn to EXPLAIN that provides the best explanation.

Testing a fix is difficult even for experienced users. To check whether an index would help, a developer can either run EXPLAIN against a large real dataset or estimate the effect by hand. The first option takes setup time, and most teams do not let junior developers run experiments on shared or production infrastructure. The second means working out how many rows a filter removes, whether the planner would use the new index, and how much cost it would save, with nothing to check the answer against. Selectivity and cardinality estimates involve detailed arithmetic that a database engine handles routinely and a person can easily get wrong. As a result, many indexing decisions are made by intuition and trial and error.

Prior research addresses parts of this problem but not the specific gap targeted here. Heuristic techniques such as multi-level indexing, partitioning, sharding, caching, query rewriting, and materialized views remain central to practical tuning, and their effectiveness is well known [1], [2], [4]. These strategies are important for a working DBA, and they are known because they have been studied for a long time. Recent research employs machine learning and reinforcement learning as techniques that help in building cost models and rewriting strategies based on the past workloads. Studies show that they provide performance improvements, but they also face training complexity and difficulties in integration [3], [6], [9], [10]. A learned optimizer can in principle detect an unusual workload and adapt to it, which static heuristics cannot, but it needs data and infrastructure that a small team or early-stage project usually lacks. A smaller body of work studies plan search itself: how algebraic and physical optimization interact [5], and, closest to this project, how what-if cost analysis can decide whether a hypothetical index is worth creating before it is built [8]. The way both professions function is such that there's nothing for a non-expert to evaluate or to learn more about. All one can get is some numbers in tables or text explaining what the optimizer has achieved.

The execution plan looks like a tree of steps with their costs, and the choice of a particular scan method, joining method, or index represents a specific step in that tree. The tree itself can be

presented and studied differently from the EXPLAIN writing. If the graphical representation of the execution plan is interactive and the possibility to turn the index on and off is provided, a student will be able to learn what is only described in theory, for example, how come hash join costed less than nested loops or how indexing one column obliges to change the entire plan while indexing another column is of no use. A junior developer can reason about a slow query without first learning to read EXPLAIN output, and a DBA or backend engineer can check an indexing decision, or explain it to a colleague, without touching a production database or waiting for a maintenance window.

The application has multiple purposes which can be used for teaching and for day-to-day activities. For example, when learning about database use, the students can modify the inputs into their formulas and see how the plan reacts to them rather than just read about the formulas on a slide without understanding it. In a working environment, the application assists with quick verification - a developer checks his hypothesis about slow queries without accessing any production data, and the DBA can present arguments for or against installing a certain index to his coworker.

This project sets its sights on a minimum viable product that can be built as opposed to a database engine and therefore possesses a narrow scope. Each of its boundaries reflects a conclusion as to where the effort of creating the project is most efficient. In practice, the system is able to execute SELECT statements that feature INNER, LEFT and RIGHT JOINs, WHEREs and ORDER BY clauses, and CREATE TABLE statements, thus successfully addressing all queries most students and beginner developers deal with in the early phase. It takes row counts from the user instead of reading real data, so it never needs a live database. Its cost engine, written in Java, uses PostgreSQL-style heuristic formulas because PostgreSQL's cost model is open, documented and widely taught. The interactive plan graph with what-if index toggles is the part of the system that prior tools do not offer. Live database connections are excluded, since they would reintroduce the risk the tool is meant to avoid. Window functions, CTEs, subqueries and GROUP BY are excluded because they would complicate the parser and cost model considerably while adding little teaching value at this stage. Only the PostgreSQL dialect is supported, since additional cost models would multiply the work without changing the approach. These limits reduce coverage, but they keep every result explainable, and they were chosen at the design stage.

1.1 Objectives

1. To study the already existing techniques to SQL optimization and checking the cost of the existing techniques
2. To identify the limitations of the already existing SQL optimization techniques, particularly with the execution visualization and the ability to safely test changes without affecting the live database
3. To design a web based simulator to visualize the approximate costs by taking in the SQL and schema input and show the result with a graph
4. To enable a safe what-if analysis by letting the user be able to select an index on a column and note down the differences between baseline and simulated costs.

2. Literature Review

There exist a variety of ways to run the same SQL query but they involve drastically different costs, which is why it is crucial to optimize the query execution. The first studies dealt with the methods of execution plan selection, statistics and heuristics, indexing, join order, and cost estimation. But with the increase in the workload size and unpredictability, the limitations of static techniques became evident and led to the emergence of adaptive, machine learning, and hybrid methods.

The present-day systems still rely heavily on their predecessors. According to Rahman et al. [1], the use of multi-tier indexing, rewriting of queries, and dynamic execution plan have greatly contributed in improving the speed and the efficiency of retrieval and execution of the task at hand as compared to their usage individually. Uzzaman et al. [2] reported their observations about the efficiency of optimization and how it can be improved according to the workload; hence, it is impossible to define which one technique works better. Ikram and Mazher [4] stated that the methods of indexing and join algorithms, transformations, optimization, materialized views, caching, etc. can be applied to the task in question. Their major argument in this case was that every method has its cost which is necessary to be paid for achieving a desired goal; an index makes the task of getting the information easier but increases the expenses associated with storage and slows down the work.

Certain researchers have experimented with employing machine learning methods to make optimizers more adaptive. Panwar [3] asserts categorically that decision trees, random forests and neural networks trained on query history can be applied for predicting the performance in terms of plan selection with admitted minimum run time, resource consumption, lower utilization of CPU and memory, and using the data available to give more accurate forecasts. However, certain costs have to be kept in mind, namely those related to computational resources, continuous support of the model used, data privacy, etc. Juopperi [7] conducted a research comparing classical optimization methods to AI-driven optimization methods introduced by novel means including the method of query reformulation applied with the help of ChatGPT and concluded that though AI can be utilized for revealing new optimization possibilities, it cannot substitute the existing methods.

The exploration of the plan search process is the focus of other studies. Gretscher and Dittrich [5] distinguish between two approaches to optimization: split optimization, where algebraic and physical optimization procedures are applied one after another, and holistic optimization, where both techniques are utilized simultaneously. Holistic optimization can obtain better plans than the split approach, but the length of its search process is significantly longer. The authors propose the hybrid top-k method that aims to retain most of the plan quality provided by holistic optimization while keeping the speed characteristic of the split technique.

The evolution from rule-based optimization to cost-based optimization and reinforcement learning is discussed by Shajeena et al. [9]. The main advantage of the latter approach is that an agent learns from execution feedback instead of relying on any fixed cost model. The implemented tool in PostgreSQL for the IMDb benchmark is able to decrease optimization costs, as well as execution

time. The authors admit that there are still open issues in terms of scalability, generalization to other systems, and real-time inference overhead.

OptimAIzerSQL [10], developed by Hettiarachchi and Yapa, separates the process between two different components: a heuristic Join Order Agent that automatically determines the proper order of the join operations and a reinforcement-learning Index Selection Agent that suggests indexes based on the features of queries and execution history. In their evaluation of their system, the joining agent improved the joining process, and the index agent provided correct index suggestions with approximately 72% accuracy. Their results were based on the IMDb and JOB datasets, testing of their approach was limited to only a few varieties of workload, and the approach was not heavily tested on dynamic schemas or dynamic workload.

Both query optimization and index selection are intertwined. With an increase in tables, the number of possible join sequences becomes combinatorial, while index selection is treated separately as an optimization method due to the fact that indexing leads to faster reads but requires more memory and slows the writing process [10]. OptimAIzerSQL solves both tasks by means of separate agents, which may be found in several other works recently published.

Optimization can involve multiple systems. Panda [6] introduces a dynamic cost-aware SQL rewriting algorithm to optimize queries in a multi-cloud environment, whose cost is not only financial but includes network transfer and platform-specific pricing as well as execution time.

Our research shares considerable ground with the work underpinning this study, primarily the work of Rahman et al. [8]. They point out that current auto-indexers do not index temporary intermediate results produced by procedural SQL (stored procedures and UDF). To overcome the limitation, they develop the concept known as an intermediate index which is based on the output columns of a producer statement but restricted to only those columns that are actually used in a consumer statement while processing data. They also develop the method called AutoCox which gives a programmer the ability to compare costs of a full-table scan and the combined cost of index creation and index scan. Since the data is temporary, the index is created right after the producer statement and dropped automatically at the program end. During testing, AutoCox demonstrated 261x speedup and almost 67% cost reduction compared to a modern baseline during experiments performed on the S-SOAP and P-TPC-DS datasets. Our what-if index enables users to benefit from AutoCox's advantages by deciding whether it is worth creating an index, but with respect to a single basic SQL query rather than to intermediate results.

In general, the studies illustrate a shift from rigid to flexible optimization processes. Even though traditional methods [1], [2], [4] are already outdated, they are adopted in practice because of their simplicity, cost-effectiveness, and high degree of interpretability. Meanwhile, modern methods ML and RL [3], [6], [9], [10] lack interpretability, and therefore they rely on real workloads. At the same time, hybrids try to combine the advantages of both approaches. As noted in the what-if analysis [8], there are examples of effective cost models that do not use any learning component. Throughout the body of literature analyzed, one can see that join order selection, index selection, query rewriting, and physical plan choice are mutually dependent, and therefore optimization of each tactic separately is less effective than in the case of a more complex workload.

2.1 Research Gap Identified

Research Gap 1: Absence of Interactive and Visual Explanation. In all ten papers analyzed, whether traditional or AI-based, performance costs and optimization decisions are presented only in tables, descriptive text and static diagrams at best. None of the papers provides an interactive option for understanding what led someone to choose a certain plan or how a modification would impact it.

Research Gap 2: AI/ML/RL Optimizers Trade Interpretability for Performance. According to research [3], [6], [9], and [10], using machine learning or reinforcement learning leads to better performance, while at the same time requiring some degree of penalties or retraining or resulting in lost insights about how decisions or plans were reached. While this tradeoff has yet to be resolved, it is one that is of particular importance to those who want to be able to explain or understand a choice.

Research Gap 3: No Safe, Data-Free Way to Test Hypothetical Indexes. AutoCox [8] is the closest system, as it utilizes what-if cost calculation in determining whether to create an imaginary index or not. It still takes a live database and actual optimizer stats and cardinality and selectivity metrics available at runtime in the procedural SQL system. Outside this context, developers and DBAs will need to perform EXPLAIN on production data which can be risky and costly or rely on manual estimations.

Summary of Research Gap. The Database Query Visual Simulator fills up the three identified gaps in the current solution. Concerning Gap 1, it has an interactive node graph to represent the cost of the query rather than static text or numbers. With respect to Gap 2, it employs an inexpensive heuristic cost model instead of a more sophisticated one, which makes it less accurate but ideally suited for an explanation and support. In regard to Gap 3, it implements the what-if method from the work [8] without actual data and a single query by allowing the user to design a hypothetical schema, provide an expected number of rows, mark any column as indexed in the hypothetical way, and see the consequences on the cost.

3. Proposed Methodology / Proposed System

3.1 Overall Methodology

The architecture follows the pipeline we have outlined in our design document. The client sends a schema and a query as plain text; the parser parses the text and constructs a model; the analyzer generates an execution plan based on the model; the cost engine computes a cost for each operation; and finally, the output of this process is sent back to the client in the form of a plan tree, which is rendered by the client as a graph. There are two architectural decisions that impact all subsequent steps. Firstly, the system does not connect to any live databases. All the costs are computed based on the user's schema and number of rows, so that the what-if toggle can be used to test a hypothesis without risking a live system. Second, the plan is built on the left-deep principle, which implies the order of tables as described in the FROM/JOIN part of the query, not the exhaustive examination of all possible orders. This way, it becomes possible to trace back each

choice made to a certain formula, which is the key goal of the explainable system. However, the price to be paid is missing the most inexpensive plan, and it was willingly taken.

3.2 System Architecture

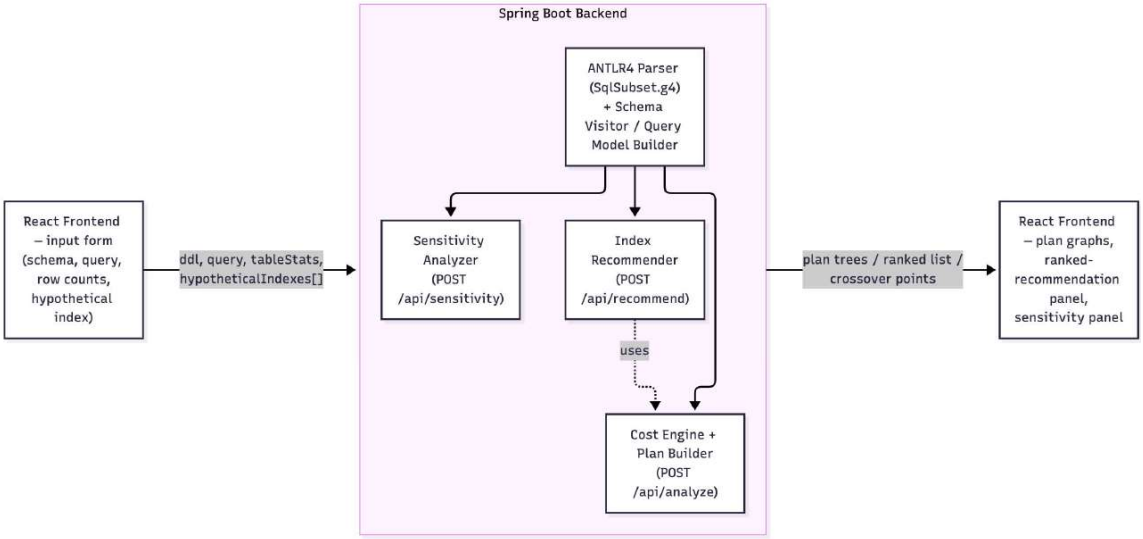


Figure 1. The frontend of React sends, in a single request to POST /api/analyze, the schema, query, row count, and an index hypothesis to the Spring Boot backend. The backend then calls the ANTLR4 parser, schema/query visitors, cost engine, and plan builder before sending back both the plan trees to the frontend.

The client app, built using React and reactflow, is a single-page application. It consists of a DDL, the query, row counts per table, an input field of a hypothetical index (table and column), and two more actions as mentioned in Section 3.5 and Section 3.6. The backend, developed in Java 21 and using Spring Boot, provides three endpoints having a common processing pipeline which includes a parser, visitors, and a cost engine. The POST /api/analyze endpoint performs analysis twice – first without indexes, and then with the hypothetical index. It returns both plan trees, together with their cost and percentage changes. As both plans are returned within one response, the frontend shows them in parallel to instantly evaluate whether the plan is changed because of the hypothetical index. The POST /api/recommend and POST /api/sensitivity endpoints have the same processing pipeline for the two analyses below.

3.3 Detailed Explanation of the Proposed Approach

Parsing. The grammar used to represent the SQL subset is contained in one file called SqlSubset.g4; it includes such SQL commands as CREATE TABLE where column data type and constraints like PRIMARY KEY, NOT NULL, UNIQUE and REFERENCES are allowed, as well as the SELECT ... FROM ... [INNER/LEFT/RIGHT JOIN ... ON ...]* [WHERE ...] [ORDER BY ...] command. Based on that grammar, ANTLR produces lexer, parser and visitor base class; two visitors work with the created parse tree in order to build a plain domain model from it. The SchemaVisitor generates a Map<tableName, TableDef>, the QueryModelBuilder builds a

QueryModel which consists of an ordered list of tables in the FROM/JOIN chain, join conditions and their types, and ORDER BY columns.

Cost estimation. The cost engine uses PostgreSQL's documented cost constants, together with the common textbook assumption of 100 rows per heap page:

Symbol	Constant	Value
C_{seq_page}	seq_page_cost	1.0
C_{random_page}	random_page_cost	4.0
C_{cpu_tuple}	cpu_tuple_cost	0.01
$C_{cpu_idx_tuple}$	cpu_index_tuple_cost	0.005
C_{cpu_op}	cpu_operator_cost	0.0025

For each base table:

$$C_{seq} = \lceil rows/100 \rceil \times C_{seq_page} + rows \times C_{cpu_tuple}$$

$$C_{idx} = \lceil rows_{matched}/100 \rceil \times C_{random_page} + rows_{matched} \times \left(C_{cpu_idx_tuple} + C_{cpu_tuple}\right)$$

C_{idx} is computed only when the filtered column is marked as hypothetically indexed, and $rows_{matched}$ is the row count after applying the selectivity estimate described below.

Selectivity, the fraction of rows a predicate matches, is estimated with a fixed rule: an equality predicate (=) matches one row, and a range predicate (<, <=, >, >=) matches 30% of the table. The engine just picks whichever scan cost is lower and reports that as the node's cost, which is also why toggling an index sometimes changes nothing. If the sequential scan is already cheaper (small table, or no filter on the indexed column), the engine sticks with it.

Joins. For each join, the engine works out two candidate strategies over the already-costed outer and inner nodes and keeps whichever is cheaper. This is really the one place the system behaves like an actual cost-based optimizer instead of a fixed heuristic:

$$C_{hash} = C_{outer} + C_{inner} + (rows_{outer} + rows_{inner}) \times C_{cpu_op}$$

$$C_{nl} = C_{outer} + rows_{outer} \times C_{probe}$$

$$C_{probe} = 2C_{random_page} + C_{cpu_idx_tuple} + C_{cpu_tuple}, if\ the\ inner\ join\ column\ is\ indexed$$

$$C_{probe} = C_{inner}, otherwise, a\ full\ inner\ scan\ per\ probe$$

Joined row count is estimated as $rows_{join} = max\left(1, min(rows_{outer}, rows_{inner})\right)$, a standard simplifying assumption for equality (foreign-key-style) joins. If the query has an ORDER BY clause, a Sort node is added at the top of the plan with cost:

$$C_{sort} = C_{child} + rows \times log_2(rows) \times C_{cpu_op}$$

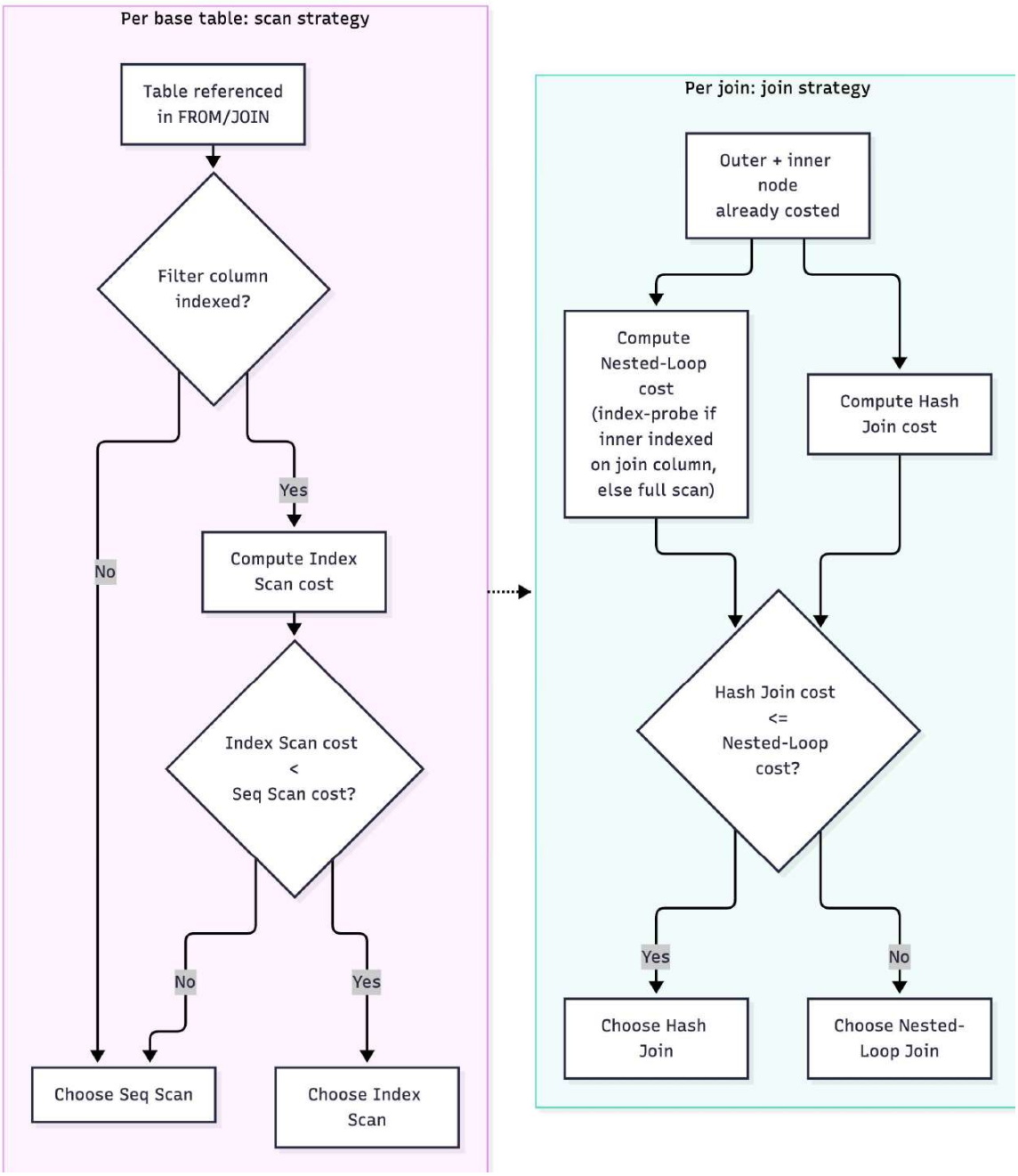


Figure 2. The two decisions the cost engine makes at each node. For a base table, it compares Seq Scan with Index Scan (only when the filtered column is indexed) and keeps the cheaper one. For a join, it compares Hash Join with (index) Nested-Loop Join and keeps the cheaper one. These comparisons are where the system behaves as a cost-based optimizer instead of following a fixed rule.

3.4 Workflow / Process of the Proposed System

1. The user enters a hypothetical schema as CREATE TABLE statements, a query, and an assumed row count for each table.

2. The user may mark one column as hypothetically indexed, then clicks **Analyze**.
3. The backend parses the schema and query with the ANTLR4 grammar and visitors, producing a TableDef map and a QueryModel.
4. The cost engine builds two plan trees from the same QueryModel: a **baseline** plan with no indexes and a **with-index** plan that includes the requested index. In each, scan and join strategies are chosen bottom-up as described in Section 3.3.
5. The backend returns both plan trees, their total costs and the percentage cost change as JSON.
6. The frontend lays out each plan tree as a node graph, with table scans as leaves from left to right and joins and sorts centred above their children, and renders both with React Flow for side-by-side comparison.

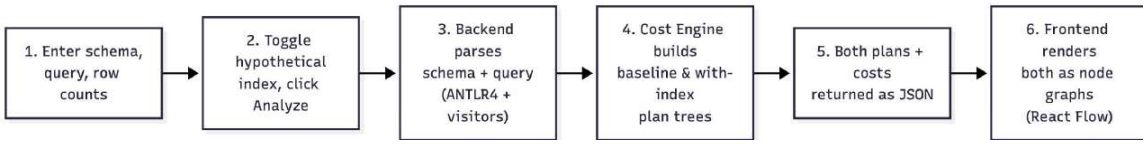


Figure 3. End-to-end workflow: entering the schema and query, toggling the index, parsing with ANTLR4, building plans with and without the index, returning the JSON response, and rendering in React Flow.

3.5 Automatic Index Recommendation

The manual what-if toggle tests one column at a time and answers whether that particular index would help. POST /api/recommend extends this to every column the query uses and identifies which index would help most. Candidate columns come from the parsed QueryModel, not from user input: every column in a WHERE predicate and both columns of every join condition, with duplicates removed. Columns declared as PRIMARY KEY are excluded, because a real database indexes them automatically and recommending them would add nothing. For all of the candidates, the engine performs the same buildPlan procedure as the manual setting of that column only in theory. Candidates are then ranked by percentage cost reduction. The ranking applies the what-if decision rule of [8] to a full set of candidate columns instead of to one column chosen in advance by the user.

3.6 Sensitivity / Crossover Analysis

The results shown in this paper will be based on the same selectivity rule as discussed in the section 3.3 where equality checks return one row and range checks give 30% of the whole table. POST /api/sensitivity makes that assumption visible by asking at what selectivity the recommended strategy changes. For a base table, the sequential-scan cost does not depend on selectivity and is held constant, while the index-scan formula from Section 3.3 is evaluated over a range of selectivity fractions. Bisection search finds the fraction at which the two costs are equal: below it the index scan is cheaper, and above it the sequential scan is cheaper. For a join, the same idea is applied to the outer table's row count instead of a selectivity fraction: holding the inner table's size fixed, the engine bisects over the outer table's row count to find where an indexed nested-loop join stops beating a hash join, the same "nested loop is only good when the outer side is small"

boundary that a real cost-based optimizer reasons about implicitly. In both situations the search is a simple binary search on a monotonic cost difference (not strictly continuous because of the page-count ceiling function) rather than an exhaustive sweep, and so each query takes well under a millisecond.

4. Implementation / Experimental Setup

4.1 Hardware and Software Requirements

Category	**Detail**
Development machine	macOS (Apple Silicon, arm64)
Backend runtime	Java 21 (OpenJDK, Homebrew build)
Backend framework	Spring Boot 3.3.4, Spring Web (embedded Tomcat)
Build tool	Apache Maven 3.9.16
Parser generator	ANTLR 4.13.1 (antlr4-maven-plugin, visitor mode)
Frontend runtime	Node.js 26.3.1, npm 11.16.0
Frontend framework	React 19 (Vite build tooling)
Visualization library	reactflow
Testing	JUnit 5 (spring-boot-starter-test)
No database required	The system has no persistence layer and no live database connection by design (Research Gap 3)

4.2 Dataset Details

There's no external dataset here, and that's by design. The entire point of the MVP is to reason about cost from a **user-supplied hypothetical schema** and **user-supplied assumed row counts**, without touching real data or a live database (Section 1, In-Scope). For development and testing we used four representative schema/query scenarios, picked to exercise each part of the cost engine:

1. A single unfiltered table scan (validates the base sequential-scan formula).
2. A single table with an equality WHERE filter, with and without an index (validates the scan-method switch).

3. A two-table equality join between a small and a large table with no index (validates hash-join selection).
4. The same join with a hypothetical index added on the large table's join column (validates the switch from hash join to index nested-loop join).

4.3 Tools, Technologies, and Frameworks Used

- **ANTLR4**, produces the lexer, parser and visitor classes for the SQL subset grammar (SqlSubset.g4) directly from a .g4 file, so a hand-written parser was not required.
- **Spring Boot**, makes the cost engine available through three REST endpoints (POST /api/analyze, /api/recommend and /api/sensitivity), while Jackson takes care of converting the plan tree and the recommendation and sensitivity results to and from JSON.
- **React + Vite**, the single-page frontend, made up of an input form for the schema, query, row counts and indexes, two panels placed side by side for the baseline and with-index plans, a ranked recommendation panel and a sensitivity panel.
- **React Flow (reactflow)**, draws each PlanNode tree as an interactive node graph that can be panned and zoomed, with the nodes colour-coded by operator type (scan, index scan, hash join, nested loop, sort).
- **JUnit 5**, holds the hand-computed cost assertions used to check the cost engine (Section 5).

4.4 Implementation Procedure

1. The ANTLR4 grammar was defined to cover CREATE TABLE and the supported form of SELECT, and the lexer, parser and visitor were then generated with the antlr4-maven-plugin.
2. The domain model (TableDef, FilterPredicate, JoinSpec, QueryModel, PlanNode, IndexRef) was built so that the rest of the system can work with it separately from the parse tree itself.
3. Implemented SchemaVisitor and QueryModelBuilder, the two visitor classes that turn a parse tree into the domain model, including alias resolution for qualified columns (t.column).
4. The CostEngine was created, which is based on the mathematical formulas for PostgreSQL as listed in Section 3.3, to construct the left-deep planning tree and determine the least expensive strategy of join and scan for each query node.
5. A Spring Boot REST controller, named AnalyzeController, was developed that encapsulates the cost engine, and runs every query two times, once without any indexes and once with the requested one, while sending back both plans.
6. Implemented IndexRecommender (Section 3.5) and SensitivityAnalyzer (Section 3.6) as further cost-engine components and exposed them through the same controller as /api/recommend and /api/sensitivity.
7. The React frontend was constructed, including an input form, a fetch call to the backend, and a layout function convertPlanNode tree into React Flow nodes and edges along with panels for recommendations and sensitivity results. Clicking a recommended column copies it into the what-if toggle.

8. Moreover, the entire process was tested, including the unit tests performed for the cost engine, index recommender, and sensitivity analyzer through the mvn test command, starting the backend process using mvn spring-boot: run, as well as the React frontend with npm run dev and testing through the browser.

4.5 Experimental Setup

The backend ran locally on port 8080 (mvn spring-boot:run) and the frontend on port 5173 (npm run dev). The frontend communicated with the backend directly through fetch, and CORS was enabled on the backend. All four scenarios from Section 4.2 were run through the automated JUnit test suite (which asserts exact cost values) as well as the browser UI (confirmed visually through screenshots, Section 5), meaning the reported numbers are verified at two independent layers instead of one.

5. Results and Discussion

5.1 Experimental Results

We ran the four scenarios from Section 4.2 through the working system; Table 1 lines up what the engine computed (verified both by the JUnit suite and by the browser UI) against the value we hand-computed from the formulas in Section 3.3.

Table 1: Cost engine output vs. hand-computed expectation

Scenario	Baseline plan (cost)	With-index plan (cost)	Cost change	Hand-computed match
Seq scan, 10,000-row table, no filter	Seq Scan (200.00)	N/A (no filter to index)	N/A	match
Equality filter on a 10,000-row table	Seq Scan (200.00)	Index Scan (4.01)	**−98.0%**	match
Equality join, 100-row × 100,000-row tables, no index	Hash Join (2252.25)	N/A	N/A	match
Same join, index on the 100,000-row table's join column	Hash Join (2252.25)	Nested Loop (803.50)	**−64.3%**	match

In all four scenarios, the engine's output matched the hand-computed cost to within floating-point rounding. In two ways this was confirmed: the CostEngineTest suite (6 test methods and 19 assertions, all passing; two of the methods test join-cardinality estimation rather than the Table 1 scenarios) and the manual recomputation of all values based on formulas from Section 3.3. Because the tests run through the parser, the domain model and the cost engine together, they check the whole pipeline and not only its individual components.

5.2 Figures

Tables 4 and 5 depict the node charts of the scenario numbers 2 and 4 respectively as mentioned in Table 1 as observed during the operation of the application. Figure 6 plots all four cost values from Table 1.

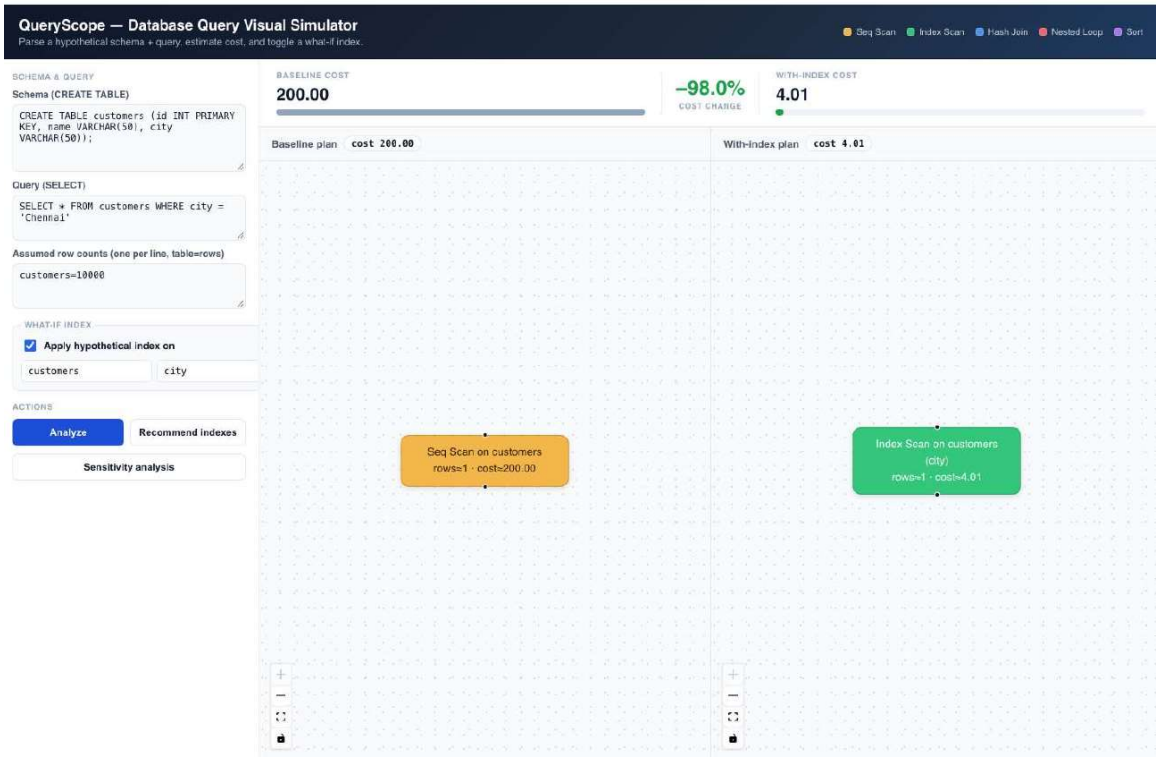


Figure 4: scan-method switch. There is a one-table equality filter employed on the customers.city table without and with a fictional index. The baseline plan shows Seq Scan node in yellow color at a cost of 200.00 whereas the indexed plan shows the Index Scan node employing the index at a cost of 4.01, corroborating the 98% decrease mentioned in the information panel.

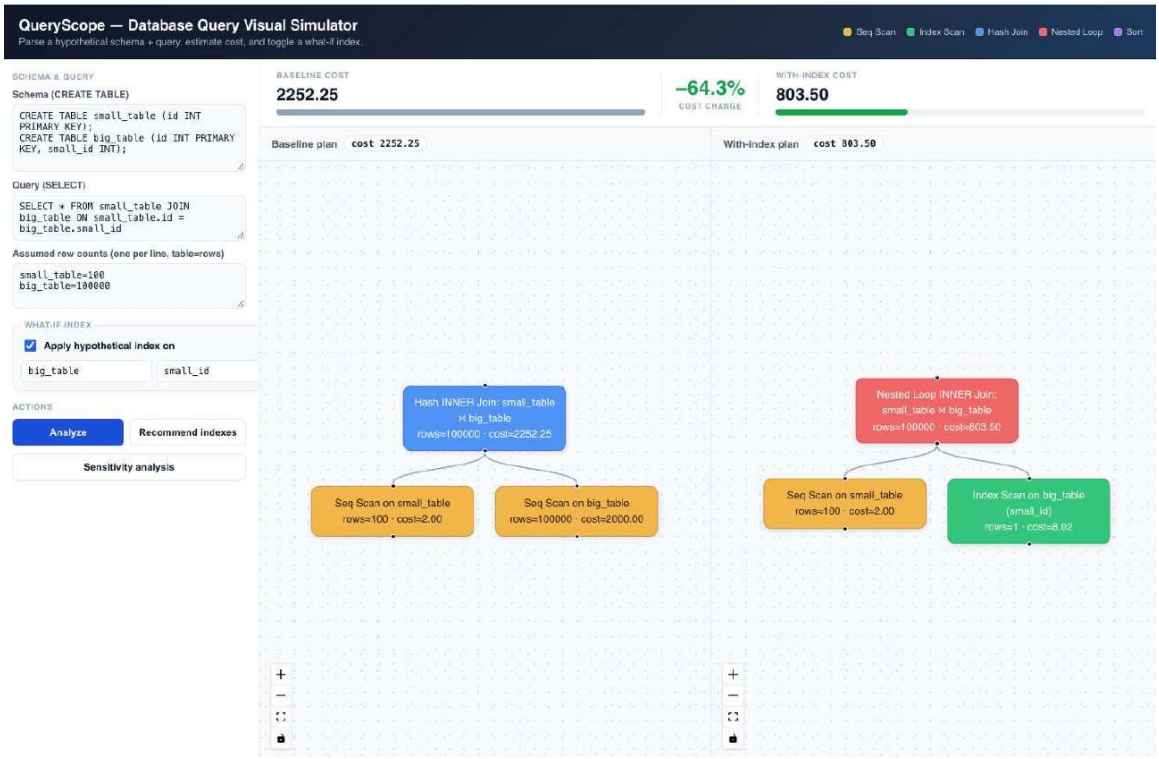


Figure 5: join-method switch. A join between a 100-row table and a 100,000-row table. The baseline plan is a blue Hash Join (cost 2252.25) over two full sequential scans. After the large table's join column is indexed, the plan becomes a red Nested Loop join (cost 803.50), and its inner child changes from a Seq Scan to a green Index Scan. The graph therefore shows the change in strategy at each level of the plan, in addition to the change in total cost.

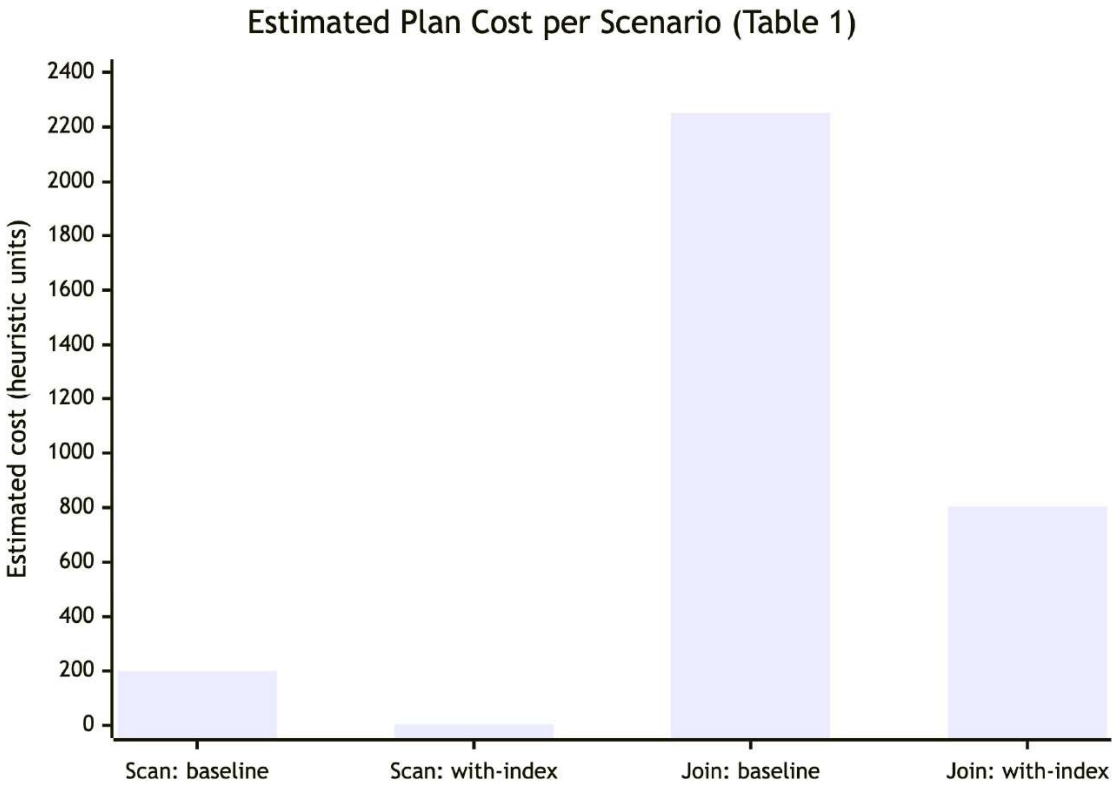


Figure 6: cost comparison between scenarios. All four cost values from Table 1. The scan scenario falls from 200.00 to 4.01 and the join scenario from 2252.25 to 803.50; both reductions are visible despite the difference in absolute scale.

5.3 Performance Metrics

The metric reported throughout is **estimated total plan cost**, expressed in the same abstract units as PostgreSQL's EXPLAIN output. We do not report wall-clock time, because no query is executed against real data, which follows from the zero-data design described in Section 1. The percentage change between the baseline and with-index plans is computed as

$$\Delta\% = 100 \times \frac{C_{baseline} - C_{with-inde}}{C_{baseline}}$$

and is displayed in the UI (Section 3.2), giving a figure that can be compared across scenarios of different absolute cost.

5.4 Comparison with Existing Methods

A direct comparison with AutoCox [8], which reports speedups of up to 261× on S-SOAP and P-TPC-DS, is not meaningful at this stage. AutoCox records the actual time taken on an actual PostgreSQL instance with real-time intermediate data unlike our system which works with no real data or database (Research Gap 3). However, both systems share a commonality in terms of the approach taken: both compare a full/sequential scan cost with that of an index so as to conclude if it is worthwhile to create an index, eventually producing the percentage improvement as an output. The 64.3% reduction in our join scenario is close to the roughly 67% cost reduction AutoCox

reports on its most reuse-heavy workloads, but the two figures come from different inputs (assumed row counts versus real database statistics) and different workloads, so this similarity should not be read as validation of our cost model.

5.5 Discussion

The results show that a transparent, formula-based cost model, with no learned component and no live database, can reproduce the two behaviours most relevant to the gaps identified in Section 2.1. The plan cost responds to a hypothetical index (Gap 3), the change is shown as a graph instead of as two figures in a table (Gap 1), and no training or retraining is required, unlike the ML and RL optimizers reviewed earlier (Gap 2). The join scenario is the more informative result, because the index changed which join algorithm the engine chose instead of only lowering the cost of the existing plan. This is the kind of change a student would otherwise have to infer from EXPLAIN output. The scan scenario shows a larger reduction (98% against 64.3%), but that figure follows directly from the assumption that an equality predicate matches a single row, and should not be taken as a general result.

5.6 Automatic Index Recommendation Results

We ran POST /api/recommend on the same small_table/big_table join from Table 1 without specifying an index. The recommender considered every column used in a filter or join condition and ranked the valid candidates:

Table 2: Ranked index candidates for small_table JOIN big_table ON small_table.id = big_table.small_id

Rank	Candidate	With-index cost	Cost change
1	big_table.small_id	803.50	**−64.3%**

We can see the same 64.3% decrease as verified in Table 1 but now we find it without specifying a column. The other joining column, small_table.id, is not given a rank because it happens to be the table's PRIMARY KEY and would therefore be indexed in an actual database. The recommender now checks each candidate's declared constraints as well as its cost effectiveness.

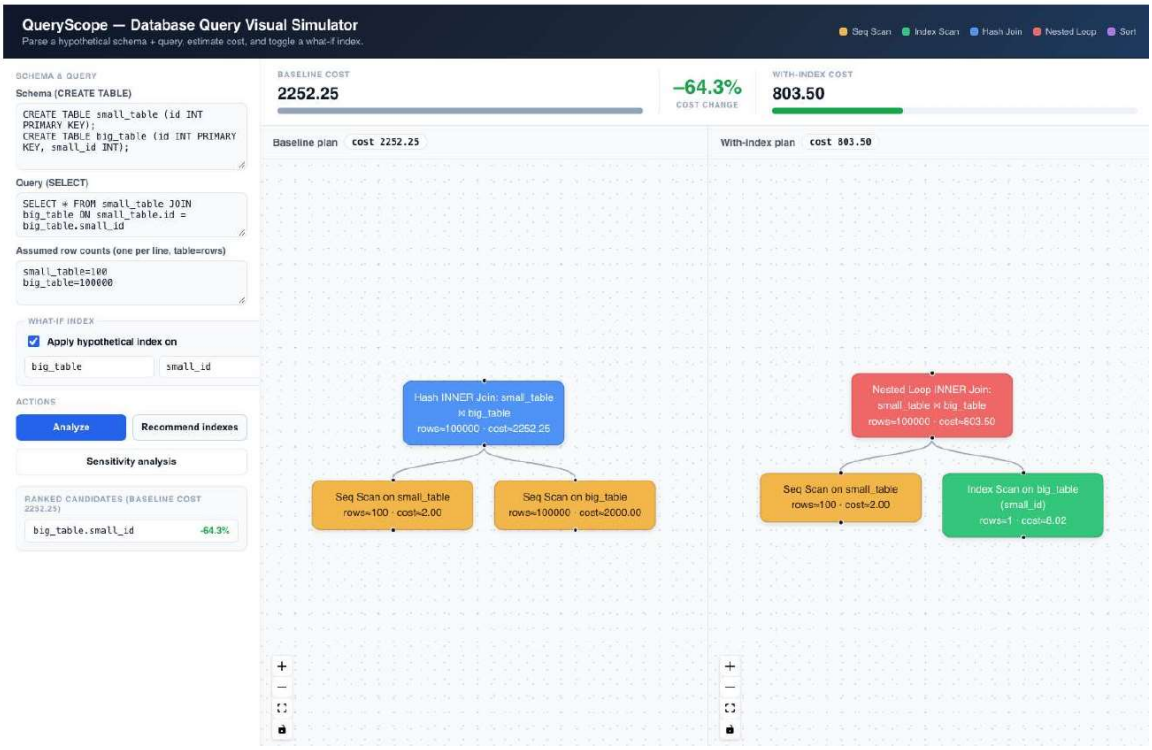


Figure 7. The ranked recommendation panel (left) and the with-index plan produced by applying the top recommendation, which matches Table 2.

5.7 Sensitivity Analysis Results

Table 3: Crossover points for the two Table 1 scenarios

Query	Crossover found	Interpretation
customers filtered on city = (10,000 rows)	Index beats Seq Scan below **~36.0%** selectivity	Our assumed selectivity for a range predicate (30%) sits just inside this boundary; an equality predicate (assumed ~0.01%) sits far inside it
small_table JOIN big_table (inner side 100,000 rows)	Indexed Nested Loop beats Hash Join below **~281** outer rows	The scenario's actual outer size (100 rows) is well inside this boundary, which is why Table 1 shows Nested Loop winning; growing small_table much past ~281 rows would flip the choice back to Hash Join

We checked both crossover values independently with a standalone Python bisection over the formulas in Section 3.3. The results matched the API output to within floating-point rounding, the same two-way check used for Table 1.

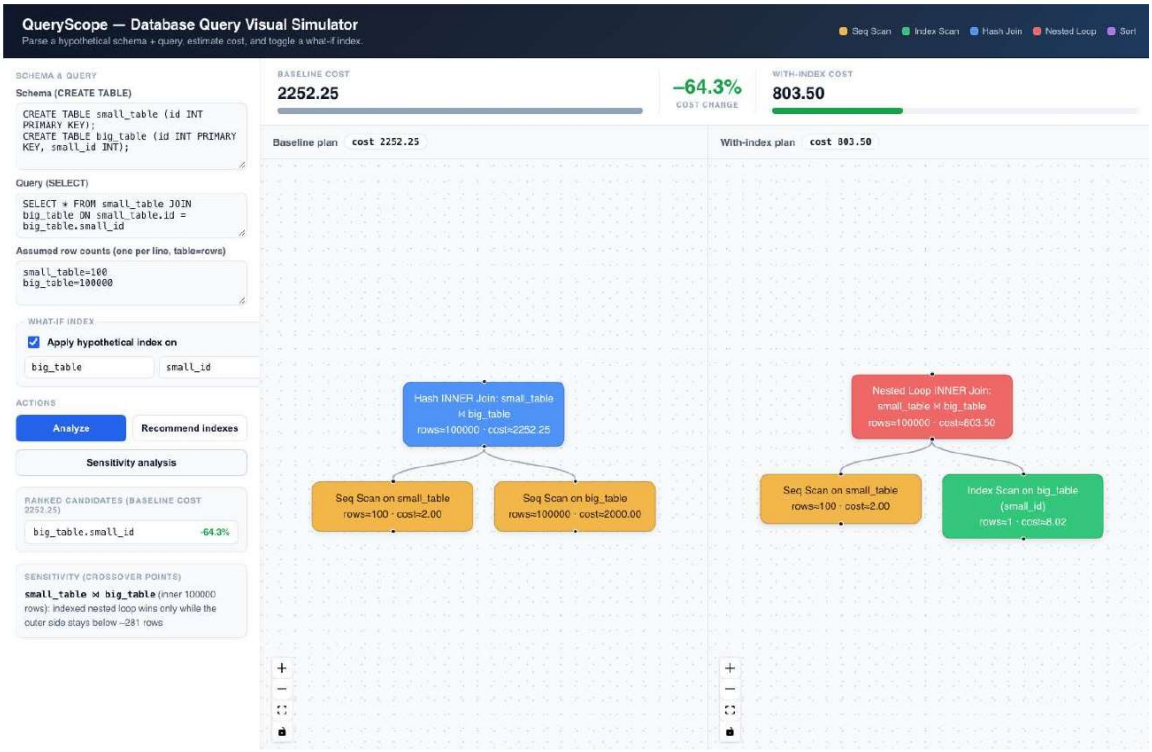


Figure 8. The sensitivity panel showing the join crossover at about 281 outer rows for the small_table/big_table scenario, alongside the resulting plan.

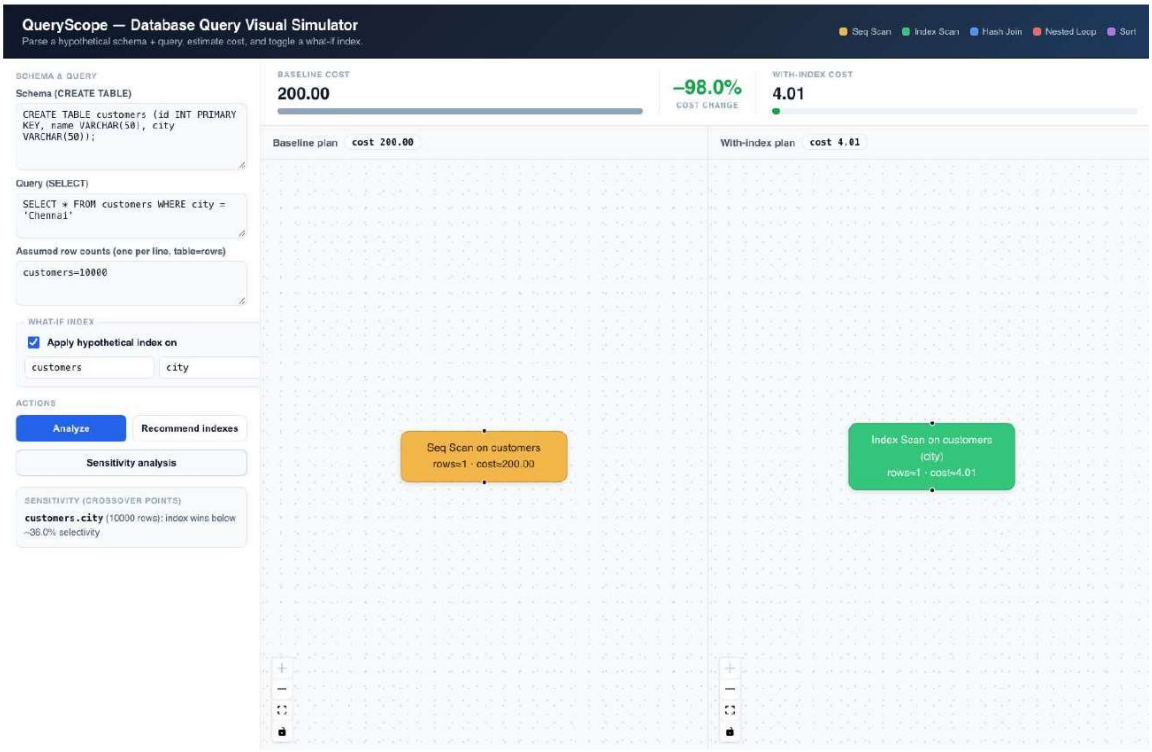


Figure 9. The sensitivity panel showing the scan crossover at about 36.0% selectivity for the customers.city scenario.

Sections 5.1 to 5.6 report the cost model's predictions under one fixed selectivity assumption. This analysis reports the point at which that assumption would change the outcome, so the user can see and reason about the selectivity rule without reading the source code.

6. Conclusion

6.1 Summary of the Proposed Work

In this undertaking, the project has dealt with an unfilled void in the field of SQL query enhancement. A situation has been created by which the decision-makers did not have a possibility of making choices, without having original data at their disposal. The Database Query Visual Simulator studies a virtual query and database with an ANTLR4 grammar, calculates costs with the help of a heuristic model based on PostgreSQL and realized in Java, and exhibits a plan for performance in the form of an interactive node network on the React platform, where both the original plan and the plan with the assumption of the existence of an index can be compared. Beyond that core toggle, the system also ranks every indexable column in a query by expected cost improvement (Section 3.5) and reports the exact selectivity or table-size boundary at which its own heuristic assumptions would flip the recommended plan (Section 3.6), turning two of the project's own limitations into results a user can inspect directly.

6.2 Key Findings / Expected Contribution

- An ANTLR4 grammar containing 2 visitor classes is sufficient to parse the MVP's subset of SQL into a structured model for cost estimation without the need for developing a custom parser.
- A small number of heuristic formulas based on PostgreSQL is able to replicate two behaviors of the optimizer, which are the choice between sequential and index scans and the choice between nested-loop and hash joins, purely based on the number of rows given by the user and the existence of a hypothetical index.
- Both behaviours appear as visible changes in the rendered node graph, not only as changes in a cost figure. This is the project's main difference from prior work (Gap 1).
- In this context, the system produces its cost estimates without relying on live data, which expands the what-if technique used in [8] to an environment where neither database nor actual and training data are required (Gap 3).
- The same what-if mechanism supports two further features. The method of obtaining candidate columns from the query and ranking them (in Section 3.5) is effective at automating the manual process of checking, while bisection of selectivity and size of the outer table (in Section 3.6) indicates to the user where the fixed selectivity assumption changes the result.

6.3 How the Identified Research Gaps Are Addressed

React Flow node graph fills **Gap 1** (no visual, interactive explanation), as demonstrated in Section 5.2. **Gap 2** (AI/ML/RL optimizers sacrificing interpretability for performance) is filled by using no learning component: all values in Table 1 can be traced back to formula (see Section 3.3). **Gap 3** (the absence of safe, data-free method of testing hypothetical indices) is filled by extending the what-if methodology described in [8] into a toggle that requires only an assumed row count and no connection with a database.

6.4 Limitations

- **Fixed selectivity rule, now bounded.** It is wrong to consider that an equality predicate corresponds to one row only, while a range predicate selects 30% of the table. Actual selectivity depends on the distribution of data, which is not known to the system. In Section 3.6 sensitiveness analysis is conducted. It does not eliminate the consideration, however, it provides the knowledge about the extent of possible deviation of actual selectivity from the assumed one before it alters the choice of the recommended plan. Such information appeals to the readers more than a figure instead of its sensitivity.
- **No join-order search.** Plans are constructed as left-deep in the order of FROM/JOIN, and the engine does not seek the least expensive join strategy as a production optimizer would do.
- **PostgreSQL-only, MVP-sized SQL subset.** Window functions, CTEs, subqueries, GROUP BY, and non-PostgreSQL cost models are all out of scope (Section 1).
- **One index at a time.** POST /api/recommend (Section 3.5) examines all candidates independently without taking any interaction into account. On the other hand, the

/api/analyze endpoint does take multiple index Neither the recommendation ranking nor the frontend currently offers a combined scenario with several indexes.

- **No calibration against a real optimizer.** Although the equations rely on PostgreSQL but have not been adapted to the actual EXPLAIN ANALYZE information from a running PostgreSQL database, so the actual costs are merely indicative rather than precise. The crossover points listed in Section 3.6 are accurate, as per the formulas used in the model, but it is not calibrated, which means that if a crossover is around 36% selectivity, then it is as reliable as the formulas used for it.

6.5 Future Scope

Utilizing cost constants allows for external confirmation of the accuracy of EXPLAIN ANALYZE results obtained from a PostgreSQL database since no connection to the PostgreSQL system would be required. The only way to do this in the past was through internal verification. By applying GROUP BY clauses and simple subqueries to the syntax, the kind of queries applicable using this technique could be enlarged. Turning the recommender from recommendations based on one column to two or three columns would suit more closely the way a DBA organizes indexes for his queries. The left-deep restriction can be overcome with a restricted join order search, even one that examines two ways of performing a two-table join and maintains the less expensive option.

CRediT authorship contribution statement:

Divyaraj: Implementation of the parser, cost engine, REST API, and frontend (Sections 3.2–3.6, 4.4); wrote and ran all unit tests and verified the Results (Sections 5.1, 5.6, 5.7).

Joel Devasia: Literature review and research gap identification (Section 2); editing and proofreading of the manuscript.

Alen Varghese Soji: System architecture and cost-model design (Sections 3.1–3.3); browser-based testing of the rendered plan graphs (Figures 1–9).

Syamasudha Veeragandham: Project Supervisor.

Funding Statement:

The authors declare that no funding, grants, or external support were received during the development of this work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

No external or real-world data was used in this study; all schema, query, and row-count scenarios used to generate the results in Tables 1–3 are given in full in Sections 4.2 and 5.1–5.7. The source code for the parser, cost engine, and frontend is available from the corresponding author upon request.

References

[1] M. M. Rahman, S. Islam, M. Kamruzzaman, and Z. H. Joy, "Advanced query optimization in SQL databases for real-time big data analytics," *Academic Journal on Business Administration, Innovation & Sustainability*, vol. 4, no. 3, pp. 1–14, Jul. 2024, doi: 10.69593/ajbais.v4i3.77.

[2] A. Uzzaman, M. M. I. Jim, N. Nishat, and J. Nahar, "Optimizing SQL databases for big data workloads: Techniques and best practices," *Academic Journal on Business Administration, Innovation & Sustainability*, vol. 4, no. 3, pp. 15–29, Jul. 2024, doi: 10.69593/ajbais.v4i3.78.

[3] V. Panwar, "AI-driven query optimization: Revolutionizing database performance and efficiency," *International Journal of Computer Trends and Technology*, vol. 72, no. 3, pp. 18–26, Mar. 2024, doi: 10.14445/22312803/IJCTT-V72I3P103.

[4] I. Ikram and N. Mazher, "Query optimization techniques in SQL: A performance-driven perspective," *Pioneer Research Journal of Computing Science*, vol. 2, no. 2, pp. 154–161, 2025.

[5] L. Gretscher and J. Dittrich, "How to optimize SQL queries? A comparison between split, holistic, and hybrid approaches," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 3910–3922, 2025, doi: 10.14778/3749646.3749663.

[6] S. P. Panda, "Dynamic cost-aware SQL rewriting algorithm for multi-cloud query optimization," in *Proc. 2025 International Conference on Intelligent Communication Networks and Computational Techniques (ICICNCT)*, 2025, doi: 10.1109/ICICNCT66124.2025.11233011.

[7] T. Juopperi, "AI-driven SQL query optimization techniques," Bachelor's thesis, Haaga-Helia University of Applied Sciences, 2024.

[8] M. A. Rahman, S. J. Hashmi, K. Gnanajothy, and Y.-K. Lee, "When temporary results meet intermediate index: An optimization technique of procedural SQL query processing," *Data & Knowledge Engineering*, vol. 164, 2026, Art. no. 102577, doi: 10.1016/j.datak.2026.102577.

[9] J. Shajeena, R. M. Shiny, K. Gayathri Devi, and M. Vespa, "A review of query optimization techniques: From traditional to reinforcement learning approaches," in *Proc. International Conference on Electronics and Renewable Systems (ICEARS)*, 2026, doi: 10.1109/ICEARS67481.2026.11416629.

[10] N. Hettiarachchi and P. Yapa, "OptimAIzerSQL: Optimizing SQL queries with heuristic and ML-based multi-agent systems," in *Proc. 5th International Conference on Machine Learning and Intelligent Systems Engineering (MLISE)*, 2025, doi: 10.1109/MLISE66443.2025.11100226.